

Your 200 lines, in four, and then in three frameworks

Part 1, in session 17: reproduce Lab 11 with scikit-learn and name every line the library replaced. Part 2, in session 18: the same network in PyTorch and Keras, started from the same weights, giving the same numbers.

Scaffolded lab

Two parts · two sessions

scikit-learn

PyTorch · Keras

Before you start

Setup · given

1 · the same model in...

2 · PyTorch and Keras, side...

Reflection

If you finish early

Before you start

Open the provided `lab12.ipynb`. Part 1 is finished in session 17 and Part 2 in session 18; each part starts from cells that are given, so Part 2 does not depend on Part 1.



By the end of this lab you can

- Map each argument of `MLPClassifier` to a line of your NumPy loop.
- Explain why a library result is close to yours but not identical.
- Build the same network in PyTorch and Keras from identical starting weights.
- Show that three frameworks produce the same loss curve when nothing differs.

Setup · given

Lab 11's data, network and a working training loop — handed out, so this lab starts from a state the whole room shares.

given

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.data / 16.0          # 1797 images, 64 pixels each,
                               # scaled to 0-1
y = (digits.target % 2 == 0).astype(float).reshape(-1, 1)  # 1 = even digit, 0 = odd digit

order = np.random.default_rng(0).permutation(len(X))
X_train, y_train = X[order[:1347]], y[order[:1347]]
X_test, y_test = X[order[1347:]], y[order[1347:]]
print("train", X_train.shape, " test", X_test.shape, " share of even digits:",
      round(y_train.mean(), 3))
```

```

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def init_params(hidden=16, seed=7):
    rng = np.random.default_rng(seed)
    return {"W1": rng.normal(0, 0.1, (64, hidden)), "b1": np.zeros((1, hidden)),
            "W2": rng.normal(0, 0.1, (hidden, 1)), "b2": np.zeros((1, 1))}

def forward(p, X):
    A1 = sigmoid(X @ p["W1"] + p["b1"])
    A2 = sigmoid(A1 @ p["W2"] + p["b2"])
    return A1, A2

def backward(p, X, y):
    # Lab 10's backward pass, unchanged
    A1, A2 = forward(p, X)
    dZ2 = (A2 - y) / len(X)
    dZ1 = (dZ2 @ p["W2"].T) * A1 * (1 - A1)
    return {"W1": X.T @ dZ1, "b1": dZ1.sum(axis=0, keepdims=True),
            "W2": A1.T @ dZ2, "b2": dZ2.sum(axis=0, keepdims=True)}

def bce(p, X, y):
    _, A2 = forward(p, X)
    return float(-np.mean(y * np.log(A2 + 1e-12) + (1 - y) * np.log(1 - A2 + 1e-12)))

def accuracy(p, X, y):
    _, A2 = forward(p, X)
    return float(np.mean((A2 > 0.5) == y))

def train(lr, batch_size, epochs, seed=1):
    p = init_params()
    rng = np.random.default_rng(seed)
    losses, accs = [], []
    for epoch in range(epochs):
        idx = rng.permutation(len(X_train))
        for start in range(0, len(X_train), batch_size):
            b = idx[start:start + batch_size]
            g = backward(p, X_train[b], y_train[b])
            for k in p:
                p[k] -= lr * g[k]
            losses.append(bce(p, X_train, y_train))
            accs.append(accuracy(p, X_test, y_test))
    return p, losses, accs

params, losses, accs = train(lr=1.0, batch_size=32, epochs=40)
print("NumPy, from Lab 11: test accuracy", round(accs[-1], 3))

```

```
NumPy, from Lab 11: test accuracy 0.976
```

the same model in scikit-learn

1

Task 1 · 8 minutes — fill the constructor

Every argument below replaces something you wrote by hand. Choose values that match Lab 11: 16 hidden units, sigmoid activation, plain mini-batch gradient descent with learning rate 1.0, batches of 32, 40 epochs, no momentum and no weight penalty.

your turn

PYTHON

```
import warnings
from sklearn.neural_network import MLPClassifier
warnings.filterwarnings("ignore")      # sklearn warns that 40 epochs "did not converge";
                                       # we chose 40 on purpose

clf = MLPClassifier(
    hidden_layer_sizes=None,           # TODO
    activation=None,                   # TODO: sklearn calls the sigmoid "logistic"
    solver="sgd",
    learning_rate_init=None,           # TODO
    batch_size=None,                   # TODO
    max_iter=None,                     # TODO: epochs
    momentum=0.0, alpha=0.0,          # plain gradient descent, no L2 penalty
    tol=0.0, n_iter_no_change=10_000,
    random_state=7,
)
clf.fit(X_train, y_train.ravel())
sk_acc = clf.score(X_test, y_test.ravel())
print("scikit-learn test accuracy:", round(sk_acc, 3), "    NumPy:", round(accs[-1], 3))
assert sk_acc ≥ 0.9
```

one version that works

PYTHON

```
import warnings
from sklearn.neural_network import MLPClassifier
warnings.filterwarnings("ignore")
clf = MLPClassifier(hidden_layer_sizes=(16,), activation="logistic", solver="sgd",
learning_rate_init=1.0,
                    batch_size=32, max_iter=40, momentum=0.0, alpha=0.0, tol=0.0,
n_iter_no_change=10_000, random_state=7)
clf.fit(X_train, y_train.ravel())
sk_acc = clf.score(X_test, y_test.ravel())
print("scikit-learn test accuracy:", round(sk_acc, 3), "    NumPy:", round(accs[-1], 3))
assert sk_acc ≥ 0.9
```

what it should print

OUTPUT

```
scikit-learn test accuracy: 0.936    NumPy: 0.976
```

Task 2 · 7 minutes — line against line

Fill the right-hand column with **where** the operation now lives, or which argument controls it.

Your Lab 11 code	In scikit-learn
init_params	
forward	
bce	
backward	
p[k] -= lr * g[k]	
rng.permutation and the batch slice	
accuracy	

Task 3 · 5 minutes — close, not identical

The two accuracies differ by a few points. Name **two** things that are different between the runs even though the architecture and learning rate match. *Hint:* print `clf.coefs_[0][:2, :3]` and `params['W1'][:2, :3]` — then think about what order the batches came in.

Your answer:

PART 2 · SESSION 18

PyTorch and Keras, side by side

2

This time **nothing** is allowed to differ: the same starting weights, the same batch order, the same loss, float64 everywhere. If the three frameworks really compute the same model, the three loss curves must lie on top of each other.



Part 2 needs `torch` and `tensorflow`. Both are installed in `atd_env`; on Colab they are already there.

given

PYTHON

```
# given - the shared starting point and the shared batch order
p_init = init_params()
rng = np.random.default_rng(1)
EPOCHS, BATCH, LR = 10, 32, 1.0
batch_order = [rng.permutation(len(X_train)) for _ in range(EPOCHS)]

def numpy_run():
    p = {k: v.copy() for k, v in p_init.items()}
    curve = []
    for idx in batch_order:
        for s in range(0, len(X_train), BATCH):
            b = idx[s:s + BATCH]
            g = backward(p, X_train[b], y_train[b])
            for k in p: p[k] -= LR * g[k]
            curve.append(bce(p, X_train, y_train))
    return curve

curve_numpy = numpy_run()
print("NumPy loss after each epoch:", np.round(curve_numpy, 4))
```

what it should print

OUTPUT

```
NumPy loss after each epoch: [0.859  0.3647 0.24   0.2916 0.5191 0.2136 0.3046 0.4143 0.172
0.1586]
```

Task 4 · 8 minutes — PyTorch

Three blanks. `nn.Linear` stores its weight as **(out, in)**, while your `W1` is (in, out) because you write `X @ W` — so one of the copies needs a transpose.

your turn

PYTHON

```
import torch
import torch.nn as nn
torch.set_default_dtype(torch.float64)

model = nn.Sequential(nn.Linear(64, 16), nn.Sigmoid(), nn.Linear(16, 1), nn.Sigmoid())
with torch.no_grad():
    model[0].weight.copy_(torch.from_numpy(p_init["W1"].T))
    model[0].bias.copy_(torch.from_numpy(p_init["b1"].ravel()))
    model[2].weight.copy_(None)          # TODO blank 1: W2, with the right orientation
    model[2].bias.copy_(None)           # TODO blank 2: b2 as a 1-D tensor

loss_fn = nn.BCELoss()                  # mean binary cross-entropy — your bce
opt = torch.optim.SGD(model.parameters(), lr=LR)
Xt, yt = torch.from_numpy(X_train), torch.from_numpy(y_train)

curve_torch = []
for idx in batch_order:
    for s in range(0, len(X_train), BATCH):
        b = torch.from_numpy(idx[s:s + BATCH])
        opt.zero_grad()
        loss = loss_fn(model(Xt[b]), yt[b])
        None                             # TODO blank 3: the one call that replaces your
backward()
        opt.step()
    with torch.no_grad():
        curve_torch.append(loss_fn(model(Xt), yt).item())
print("PyTorch:", np.round(curve_torch, 4))
```

one version that works

PYTHON

```
# (blank 1) model[2].weight.copy_(torch.from_numpy(p_init["W2"].T))
# (blank 2) model[2].bias.copy_(torch.from_numpy(p_init["b2"].ravel()))
# (blank 3) loss.backward()
```

Task 5 · 6 minutes — Keras

Keras's `Dense` stores its kernel as **(in, out)** — the same orientation as yours, so no transpose. Two blanks.

your turn

PYTHON

```
import tensorflow as tf
tf.keras.backend.set_floatx("float64")

kmodel = tf.keras.Sequential([tf.keras.Input(shape=(64,)),
                              tf.keras.layers.Dense(16, activation="sigmoid"),
                              tf.keras.layers.Dense(1, activation="sigmoid")])
kmodel.set_weights(None)  # TODO blank 1: a list of four arrays, in layer order
kmodel.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=LR),
               loss="binary_crossentropy")

curve_keras = []
for idx in batch_order:
    for s in range(0, len(X_train), BATCH):
        b = idx[s:s + BATCH]
        None  # TODO blank 2: one call that runs forward, backward
and update on this batch
        curve_keras.append(float(kmodel.evaluate(X_train, y_train, verbose=0)))
print("Keras: ", np.round(curve_keras, 4))
```

one version that works

PYTHON

```
# (blank 1) kmodel.set_weights([p_init["W1"], p_init["b1"].ravel(), p_init["W2"],
p_init["b2"].ravel()])
# (blank 2) kmodel.train_on_batch(X_train[b], y_train[b])
```


Task 6 • 6 minutes — three curves, one line

Given. Run it and read the last line.

given

PYTHON

```
plt.plot(curve_numpy, "o-", label="NumPy")
plt.plot(curve_torch, "s--", label="PyTorch")
plt.plot(curve_keras, "x:", label="Keras")
plt.xlabel("epoch"); plt.ylabel("training loss"); plt.legend(); plt.title("One model, three frameworks")
plt.show()
print("largest NumPy-PyTorch gap:", f"{np.max(np.abs(np.array(curve_numpy) - curve_torch)):.1e}")
print("largest NumPy-Keras gap:  ", f"{np.max(np.abs(np.array(curve_numpy) - curve_keras)):.1e}")
```



 **What the gaps should be.** NumPy and PyTorch agree to about $1e-12$ — the same arithmetic, in a different order. Keras lands within about $1e-6$: its cross-entropy clips probabilities at $1e-7$ to avoid $\log(0)$, a detail your `bce` handles with `1e-12` instead. A gap of 0.01 or more means one of the blanks is wrong, most often a missing transpose.

Reflection



Write it in the notebook, in your own words

`loss.backward()` replaced the function you derived by hand in session 14. Why did that one line not feel like magic today — and what would a student who started with PyTorch be unable to check?

Your answer:

If you finish early

A. Replace `torch.optim.SGD` with `torch.optim.Adam(model.parameters(), lr=0.01)` and plot the new curve against the old one. Session 19 explains what Adam is doing.

your turn

PYTHON

```
# TODO (optional extension A)
```

CSC 4315 — Artificial Intelligence · Lab 12 · accompanies session 17–18 · Fall 2026

Prof. Guillaume Sanje Mpacko, Ph.D. · School of Science and Engineering · International University of Grand-Bassam